

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Eléctrica
IEE2463 Sistemas Electrónicos Programables

Ayudantía 8: Boot y Debug con Vitis

Ayudante: José Agustín Milling - ja.milling@uc.cl

Profesor: Michel Rozas Fernández - mvrozas@uc.cl

---

1. Objetivo de la Ayudantía

- Aprender a cargar un código de BOOT a la Zybo programando la memoria flash.
- Comprender cómo usar el modo Debug en Vitis.
- Ejercitar el uso de funciones, macros y arreglos.

2. Actividades Previas a la Ayudantía

En el video de la ayudantía se explica y desarrolla el hardware y código C utilizados, además del uso del modo Debug en Vitis y cómo cargarle un código de BOOT a la Zybo programando su memoria flash.

- Crear un nuevo proyecto en Vivado asociado a la ZyboZ7-10 y en el mismo un Block Design donde se conectarán los módulos Zynq7 Processing System y AXI Gpio de acuerdo a lo visto en el video de la ayudantía. El diagrama de bloques final se muestra en los anexos.
- Incluir archivo Constraints asociado a la tarjeta y editarlo de acuerdo al Block Design desarrollado.
- Generar el HDL Wrapper del Block Design y luego el bitstream, para exportar el hardware y luego abrir Vitis desde Vivado.
- Una vez en Vitis crear un Platform Project con el archivo xsa generado y buildear. Tras esto, crear un Application Project vacío, luego en el mismo en la carpeta src haciendo click derecho añadimos un archivo C, el cual se corresponderá con el archivo `boot_sw.c` disponible en el repositorio. Luego buildear nuevamente.

- **Debuggeo:** En Vitis vaya a Debug (esquina superior derecha, al lado de Design) y clickear. Una vez en el modo Debug, vaya a la ventana Explorer en la esquina inferior izquierda y clickeando sobre Boot SW System → Debug As → 1 Launch Hardware. Tras lo anterior ya debería estar andando y debería visualizar algo como la imagen que se muestra a continuación.

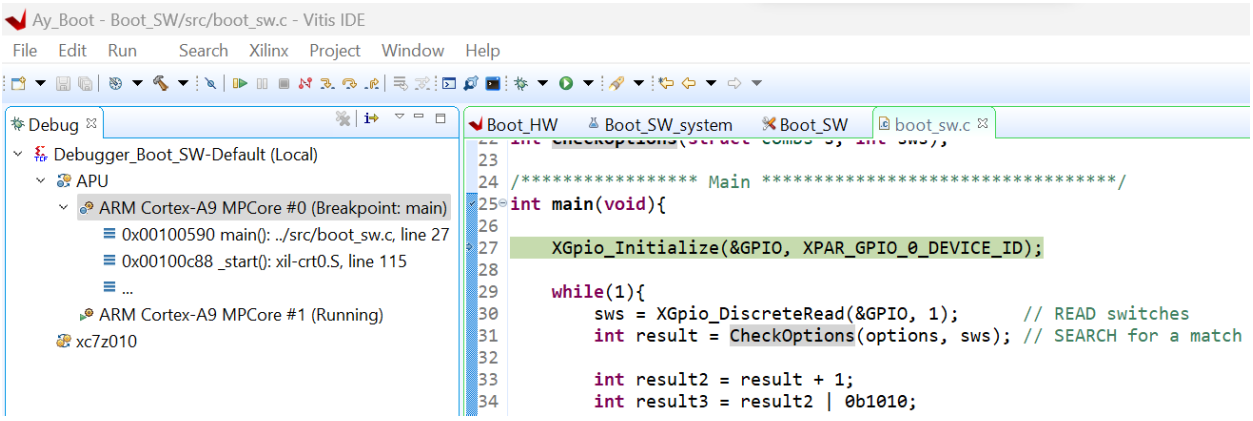


Figura 1: Vista al debuggear correctamente en Vitis

- El debuggeo y la adición de expresiones para su visualización se explican en el video de la ayudantía, pero tenga en consideración que con la flecha Step Over (F6) debería poder ir avanzando línea por línea en el código. Finalmente, cabe mencionar que generalmente hay varias opciones al seleccionar Debug As. En esta ayudantía se usa una de forma arbitraria, pero pueden probar con otras formas y no debería haber problema.

- **Booteo:** Para generar el archivo BOOT.bin necesario para programar la memoria flash de la Zybo, considerando el código desarrollado en la ayudantía, debe seguir los siguientes pasos:

  1. Build Project
  2. Click derecho sobre Boot SW → Create Boot Image
  3. Una vez en el wizard respectivo, añadir los archivos y settear los parámetros de acuerdo a lo visto en la ayudantía.
  4. Xilinx → Program Flash → Program, cuidando que Flash Type se encuentre en la opción qspi-x4-single.
  5. Una vez programada la memoria flash de la Zybo, desconéctela del PC (desenergícela) y cambie el tope azul en JP5 de JTAG a QSPI. Tras ello vuelva a alimentar la Zybo. Debería ver en funcionamiento la secuencia de BOOT que se trabajó en la ayudantía y cargó a la memoria flash, pudiendo mover los switches y ver cómo cambian los leds en base a ello.

3. Actividades Durante la Ayudantía

El ejercicio propuesto consiste en convertir la Zybo en una caja fuerte: se abre solo si se ingresa, con los switches, una secuencia secreta de combinaciones. Primero se ejercita el modo Debug, luego se programa la caja y finalmente se graba en la memoria flash para que funcione sin PC. Se utiliza exactamente el mismo hardware desarrollado previamente, por lo que no es necesario volver a Vivado.

Parte A: Debug (código original)

Sin modificar `boot_sw.c`, entre al modo Debug, ponga un *breakpoint* en la línea `XGpio_DiscreteWrite(&GPIO, 2, result3);` y agregue como *Expressions* las variables `sws`, `result`, `result2` y `result3`. Para cada combinación de switches de la tabla presione Resume (F8), deténgase en el breakpoint y complete:

sws	`result`	`result2`	`result3` (binario)	LEDs observados
`0b0000`				
`0b0101`				
`0b0111`				
`0b1111`				

a) ¿Qué hace realmente la función `CheckOptions`? ¿Es posible que retorne -1 con este hardware? Justifique.

b) Con `sws = 0b1111`, ¿cuánto vale `result3` en el debugger y qué se ve en los LEDs? Explique la diferencia.

c) ¿Qué LEDs nunca se apagan? ¿Cuántos patrones distintos de LEDs puede producir este programa?

Parte B: Caja fuerte

Cree un nuevo archivo C (puede partir desde `boot_sw.c`) que cumpla con el siguiente comportamiento:

- La clave es una secuencia de 4 combinaciones de switches guardada en un arreglo, por ejemplo `{0b1000, 0b1100, 0b0110, 0b0011}` (dos pasos consecutivos deben ser distintos).
- Cuando los switches cambian, el programa espera 1 s (para que alcance a mover varios switches) y lee el valor final: ese es el *intento*.

- Si el intento es correcto, los LEDs muestran el avance como una barra: `0001`, `0011`, `0111`, `1111`.
- Si el intento es incorrecto, los 4 LEDs parpadean rápido 3 veces y se vuelve a empezar desde el primer paso.
- Al completar la clave, los 4 LEDs parpadean lento 5 veces y la caja se vuelve a cerrar.

Requisitos de implementación:

- **Macros:** defina con `#define` el largo de la clave, los tiempos y los canales de la GPIO.
- **Arreglos:** use un arreglo para la clave y otro para los patrones de la barra de progreso, de modo que `progreso[indice]` sea lo que se escribe en los LEDs.
- **Funciones:** implemente y use las siguientes:

```
int  LeerIntento(int anterior);  // espera un cambio y retorna el valor final
void Parpadear(int patron, int veces, int semiperiodo_ms);
```

Indicaciones:

- Para usar `usleep` incluya `"sleep.h"`.
- Guarde en una variable `indice` cuántos pasos correctos lleva; el paso que se debe comparar es `clave[indice]`.
- Si algo no funciona como espera, use el modo Debug agregando `indice`, `intento` y `clave[indice]` como Expressions.
- Tras la edición → Build Project. Cárguele el código a la Zybo por JTAG como normalmente lo hace y verifique su funcionamiento.

Parte C: Booteo de la caja fuerte

- Genere un nuevo BOOT.bin con la caja fuerte y programe la memoria flash siguiendo los pasos de la sección de Booteo.
- Desenergice la Zybo, cambie JP5 a QSPI y vuelva a alimentarla: la caja fuerte debe funcionar sin el PC conectado.
- ¿Qué tres archivos componen el BOOT.bin y en qué orden deben ir? ¿Por qué el FSBL debe ser el primero?

Anexos

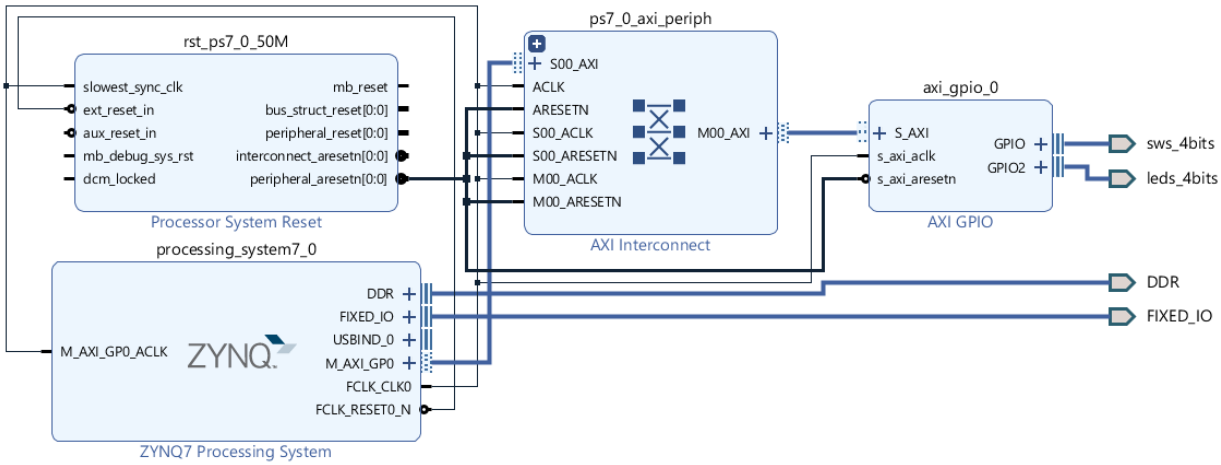


Figura 2: Block Design en Vivado del Proyecto